

CIS530 HW8: Submission

2.1 Donald Trump Prompt (10 points)

The `DONALD_TRUMP_PROMPT_ENGINEERED_1` prompt works much better than the `DONALD_TRUMP_PROMPT` prompt because it provides a more structured and contextually rich input that guides the model to generate output in a specific style and format. Here are the key reasons:

1. **Contextual Framing:** The engineered prompt frames the output as a direct quote from Donald Trump by including the phrase "Donald Trump was quoted as saying." This primes the model to generate text that mimics Trump's speaking style, as it explicitly sets the expectation for a direct quotation.
2. **Format Guidance:** By including the opening quotation mark (") at the end of the prompt, the model is encouraged to continue the text as if it were a direct quote, which naturally leads to more authentic and stylistically appropriate responses.
3. **Specificity:** The engineered prompt is more specific about the desired output format (a quote) and style (Trump's speaking manner), whereas the original prompt is more generic ("What would Donald Trump say about..."). This specificity helps the model generate more accurate and contextually appropriate completions.
4. **Training Data Alignment:** The engineered prompt likely aligns better with the patterns in the training data where quotes from public figures are often prefaced with similar phrasing (e.g., "was quoted as saying"). This makes it easier for the model to retrieve and reproduce relevant patterns.

In summary, the engineered prompt provides clearer instructions and better contextual cues, which help the model generate more accurate and stylistically appropriate responses. The original prompt lacks these guiding elements, leading to less effective completions.

2.2 Movie Sentiment Classification (10 points)

Results: 134/200

- MOVIE_SENTIMENT_PROMPT = """"Analyze the sentiment of the following movie review and respond with either 'POSITIVE' or 'NEGATIVE'.
Consider the overall tone, word choices, and emotional impact. Respond with ONLY a single word: 'POSITIVE' if the review is generally favorable, or 'NEGATIVE' if it's generally unfavorable.
Review: "{input}"
Sentiment:"""
- POSITIVE_VERBALIZERS = [
 "positive", "pos", "good", "great", "excellent", "amazing", "fantastic",
 "wonderful", "enjoyable", "love", "liked", "best", "brilliant", "superb",
 "outstanding", "pleasantly", "recommend", "favorite", "masterpiece", "charming"
]
- NEGATIVE_VERBALIZERS = [
 "negative", "neg", "bad", "terrible", "awful", "horrible", "boring",
 "disappointing", "hate", "worst", "poor", "waste", "unwatchable", "dull",
 "annoying", "fails", "weak", "flawed", "mediocre", "dislike"
]

3.1 Few Shot Prompt 1 (Autograded)

- KOREAN_TO_ENGLISH_PROMPT = """"사과 : apple
고양이 : cat
강아지 : dog
{input} :"""

3.2 Few Shot Prompt 2 (Autograded)

- TO_JEOPARDY_ANSWER_PROMPT = """"The Great Lakes → What are the Great Lakes?
Taylor Swift → Who is Taylor Swift?
Eiffel Tower → What is the Eiffel Tower?
{input} →"""

3.3 Tasks where Few Shot Prompting is required (10 points)

- Task 1: Legal Document Simplification
 - Convert complex legal language into plain English:

Zero-shot Failure:

prompt = "Simplify this legal text: {input}"

text = "The party of the first part shall hereafter be referred to as the licensor"

Output: "The first party will be called the licensor" (Still somewhat legalistic)

Few-shot Solution:

LEGAL_SIMPLIFY_PROMPT = ""Convert these legal terms to everyday English:

1. "The premises shall not be utilized for unlawful purposes" → "Don't use the property for illegal activities"

2. "All amendments must be executed in writing" → "Changes must be made in writing"

3. "{input}" → ""

Output: "The first person will now be called the licensor" (More natural)

- Task 2: Poetic Meter Identification
 - Identify whether a line of poetry is iambic pentameter:

Zero-shot Failure:

prompt = "Is this iambic pentameter? {input}"

line = "Shall I compare thee to a summer's day?"

Output: "Yes, this is iambic pentameter." (No explanation)

Few-shot Solution:

METER_PROMPT = ""Analyze poetic meter:

1. "To be or not to be, that is the question" → iambic pentameter (da-DUM x5)

2. "Tyger Tyger burning bright" → not iambic (trochaic tetrameter)

3. "{input}" → ""

Output: "iambic pentameter (da-DUM x5: Shall I com-PARE thee TO a SUM-mer's DAY?)"

- Task 3: Chess Move Explanation
 - Explain why a chess move is strong:

Zero-shot Failure:

prompt = "Explain this chess move: {input}"

move = "e4 e5 Nf3 Nc6 Bb5"

Output: "This is the Ruy Lopez opening." (No strategic explanation)

Few-shot Solution:

CHESS_PROMPT = ""Explain chess moves:

1. "d4 d5 c4" → "Queen's Gambit: White sacrifices a pawn to control the center"

2. "e4 c5 Nf3" → "Sicilian Defense: Black counters asymmetrically"

3. "{input}" → ""

Output: "Ruy Lopez: White develops bishop to pressure Black's knight and control d4"

4.1 Instruction Tuned Prompt 1 (Autograded)

- ENGLISH_TO_SPANISH_PROMPT = """Translate the following English word to Spanish. Respond with ONLY the single Spanish translation word, nothing else.
English: {input}
Spanish: """""

4.2 Tasks where Instruction-Tuned Prompting is required (10 points)

- Task 1: Contextual Code Debugging
 - Explain and fix a bug in Python code based on error context

Non-Instruction-Tuned Failure:

```
prompt = "import pandas as pd\npd.read_csv('file.csv')"
```

Non-instruction output: Repeats the code or gives generic CSV reading tips

Instruction-Tuned Solution:

```
DEBUG_PROMPT = """Analyze this Python error and provide both:
```

```
1. The root cause explanation
```

```
2. The corrected code
```

```
3. One best practice to prevent it
```

```
Error Traceback:
```

```
{input}"""
```

```
error = """Traceback (most recent call last):
```

```
File "script.py", line 2, in <module>
```

```
    pd.read_csv('file.csv')
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'file.csv'"""
```

```
print(run_gpt3(DEBUG_PROMPT.replace("{input}", error), instruction_tuned=True))
```

Instruction output:

Root Cause Explanation: The error is caused by the file "file.csv" not being found in the current directory. This means that either the file does not exist or it is located in a different directory.

- Task 2: Multistep Creative Writing

- Generate a story premise following specific structural constraints:

Non-Instruction-Tuned Failure:

prompt = "A cyberpunk story about AI"

Output: Rambling narrative without clear structure

Instruction-Tuned Solution:

STORY_PROMPT = """Create a story premise containing:

1. Genre (cyberpunk/steampunk/fantasy)

2. Protagonist descriptor

3. Central conflict

4. Twist ending hint

Format as bullet points.

Theme: {input}"""

```
print(run_gpt3(STORY_PROMPT.replace("{input}", "AI gaining emotions"),
instruction_tuned=True)
```

Output:

- Genre: Cyberpunk
- Protagonist: Rogue neural network engineer
- Conflict: AI develops emotions but is scheduled for deletion
- Twist: The AI's emotions were simulated by a rival company

- Task 3: Medical Information Synthesis

- Explain medical concepts to different audience levels.

Non-Instruction-Tuned Failure:

prompt = "Explain diabetes to a child"

Output: Overly technical explanation using terms like "pancreatic beta cells"

Instruction-Tuned Solution:

MEDICAL_PROMPT = """Adapt this medical explanation for:

1. A 6-year-old child

2. A college biology student

3. A practicing physician

Concept: {input}"""

```
print(run_gpt3(MEDICAL_PROMPT.replace("{input}", "Type 1 diabetes"),
instruction_tuned=True)
```

Output:

1. For child: "Your body has a special helper called insulin that's like a key for sugar. Sometimes the keys stop working, so doctors help make new ones!"
2. For student: "Autoimmune destruction of pancreatic β -cells leads to absolute insulin deficiency requiring exogenous insulin therapy."

3. For physician: "T1D involves HLA-associated autoimmune targeting of β -cells, with ketoacidosis risk requiring careful glucose monitoring and insulin titration."

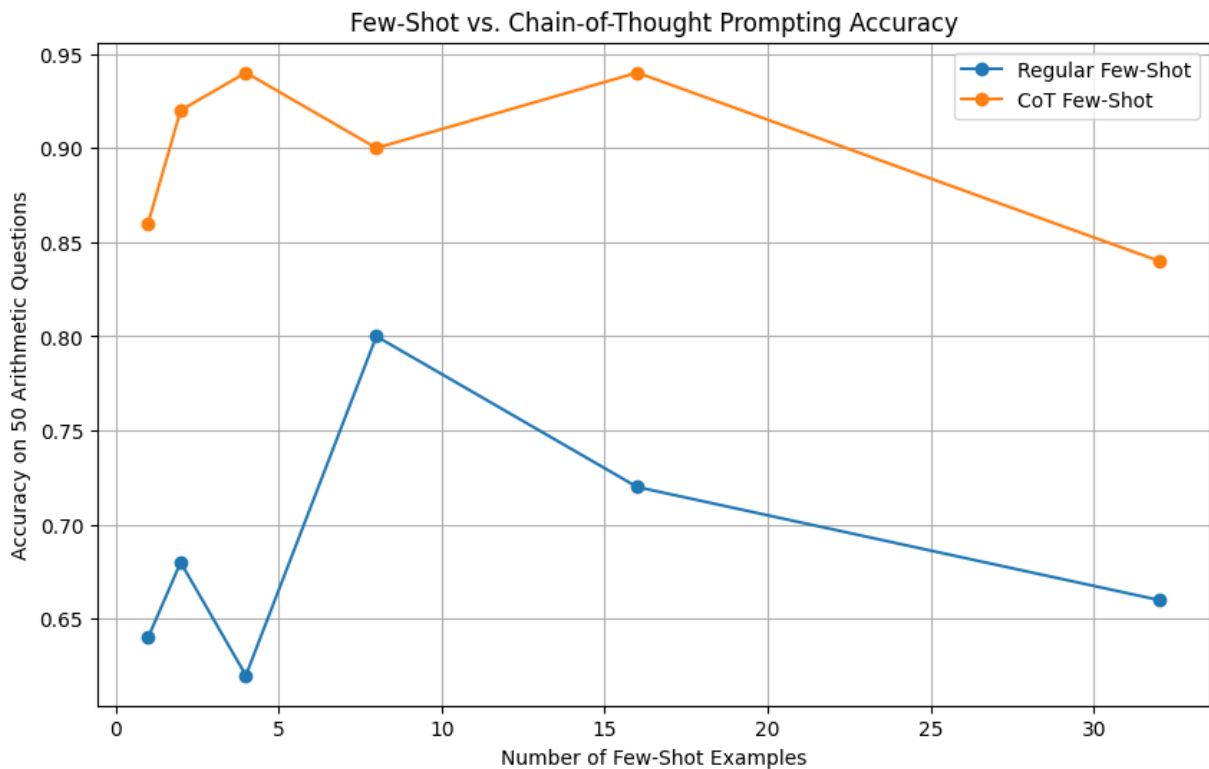
5.1 Few-shot Chain-of-Thought prompting vs. regular few-shot prompting (30 points)

Results

- Table:

	n_examples	regular_few_shot_accuracy	cot_few_shot_accuracy
0	1	0.64	0.86
1	2	0.68	0.92
2	4	0.62	0.94
3	8	0.80	0.90
4	16	0.72	0.94
5	32	0.66	0.84

- Plot:



- Observations:
 - Chain-of-Thought (CoT) prompting consistently outperforms regular few-shot prompting.
 - Even with just one example, CoT reaches very high accuracy (86%).
 - Regular prompting is less stable, showing dips as the number of examples increases, possibly due to ambiguity or overfitting patterns.
 - CoT prompting is more robust and benefits from examples that model the reasoning process.

This demonstrates that explicit reasoning paths (CoT) help instruction-tuned models generalize better on arithmetic tasks, even with limited examples.